# On the Origin of Objects

By Means of Careful Selection

YEGOR BUGAYENKO, Huawei, Russia
MAXIM TRUNNIKOV, Huawei, Russia

We introduce a taxonomy of objects for the EO programming language. This taxonomy is designed with a few principles in mind: non-redundancy and simplicity. The taxonomy is supposed to be used as a navigation map by EO programmers. It may also be helpful as a guideline for designers of other object-oriented languages or libraries for them.

Additional Key Words and Phrases: Object-Oriented Programming

## 1 Introduction

Each object-oriented programming language offers its own set of objects, classes, types, functions, constants, traits, templates, and so on, which programmers can use off-the-shelf to model their specific use cases: "Development Kit" in Java [18, 21], "Standard Library" in C++ [8, 14], Python [12, 19], Swift [4, 10], Rust [5, 20], and Ruby, ".NET Standard" in C# [3, 16], and "Built-in Objects" in JavaScript [9, 17].

Each standard library (SL) freely defines its own principles of abstraction. For example, to get the absolute value of a numeric object `x` in Java, one has to call a static method of another class `Math.abs(x)`, while in Ruby it would be a property of the same object `x.abs`. For example, in Java, an attempt to get a value from a hash map by a key will produce `null` in case of its absence, while in C++ it will return an empty iterator. There are many similar examples demonstrating the absence of a common ground between SLs.

Earlier, Booch and Vilot [6] suggested their own components for OOP. We suggest our own taxonomy of objects for EO[1], a strictly formal [15] object-oriented programming language with an intentionally reduced feature set [7]. All objects are grouped as such:

- Bytes (Section 3)
- Boolean (Section 4)
- Number (Section 5)
- String (Section 6)
- Tuple (Section 7)
- Error handling (Section 8)
- Flow control abstractions (Section 9)
- Non-standard bit-width numbers (Section 10)
- Memory abstractions (Section 11)
- Math functions and algorithms (Section 12)
- Text manipulations (Section 13)
- Structs (Section 14)
- I/O Streams (Section 15)

[1] https://www.eolang.org, 9.9.9

- File System abstractions (Section 16)
- Net Sockets (Section 17)
- System and OS abstractions (Section 18)

An object in EO is either a composition of other objects or an atom, which is a foreign function interface to a lower-level runtime such as, for example, JVM or Assembly. In this paper, we denote atoms by a star, for example: ⋆`times`.

Throughout the paper, we use dark blue color to highlight the names of objects, as opposed to other pieces of fixed-width text.

In this paper[2] we don't provide exact specification of each object, in order to avoid conflicts with their exact definitions in the Objectionary[3].

## 2 Principles

We tend to minimize the global scope, keeping the number of global objects to the absolute possible minimum. We treat objects as the words of an OOP prose: the objects that reside at the top level of the root package and its sub-packages are the nouns of its vocabulary. The more concise this vocabulary is and the less ambiguous, the cleaner the prose written with it. Synonyms are especially harmful here, since two names for the same meaning make the prose more artistic but less strict, forcing the reader to guess which one carries the intent. We therefore select global objects carefully, making sure that none of them duplicate each other, and that each delivers a clear and distinct meaning, favoring **preciseness over expressiveness**.

Wherever possible, objects must be implemented in EO, instead of being atoms. An atom is a function implemented by the runtime beneath EO, by the virtual machine or the CPU itself, and therefore runs genuinely faster than its EO counterpart. An atom is also opaque, though: neither the EO compiler nor any static analyzer can look inside it, reason about it, or optimize it. A program saturated with atoms looks like a solid brick to these tools— it either works as it happens to work, or nothing can be done about it. We instead want as many objects as possible to stay transparent to the compiler, which we expect to reason about them more powerfully than the programmers of atoms ever could. Eventually, we expect an OO program written in EO, resting on only a handful of atoms, to be optimized down to highly efficient assembly code. We therefore favor **transparency over speed**.

There should be no functionality implemented two or more times in the entire taxonomy. We treat our objects as the building blocks of larger OOP systems, and we want every block to be used and to be known to the EO compiler. A programmer who reaches for them should never need to reinvent the wheel: we intend to supply everything, from array sorters and regular expression processors to file readers and writers. The more reusable each object is, the weaker the temptation to implement something "better" elsewhere, and the fewer the private duplicates scattered across programs. We accept that a few of our objects may grow complex, or

[2] LaTeX sources of this paper are maintained in the REPOSITORY GitHub repository, the rendered version is `0.0.0`.

[3] https://github.com/objectionary/home

carry functionality that feels redundant, and that is fine. Later, once the EO compiler becomes powerful enough, it will know how to reduce that complexity and optimize it away at compile time. We therefore favor **reusability over complexity**.

## 3 Bytes

The center of the taxonomy is the `bytes` object, which is an abstraction of a sequence of bytes. The sequence can either be empty or contain a theoretically unlimited number of bytes. A byte is a unit of information that consists of eight adjacent binary digits (bits), each of which consists of a `0` or `1`. In EO syntax, a sequence of bytes is denoted as either `--` (double dash) for an empty one or as a concatenation of `xx-`, where `xx` is a hexadecimal representation of a byte. For example, the `2A-` is a one-byte sequence, where the only byte equals to the decimal number 42.

The `bytes` objects has a few attributes for bitwise operations: `⋆and`, `⋆or`, `⋆xor`, `⋆not`, `⋆right` and `left`.

The object also has the `⋆eq` attribute for byte-by-byte comparing itself with another sequence of bytes.

The `⋆size` attribute is the total number of bytes in the sequence.

The `⋆slice` attribute represents a subsequence inside the current one.

The `⋆concat` attribute is a new sequence, which is a concatenation of two byte sequences: the current and the provided one.

## 4 Boolean

The `bool` object abstracts a boolean value, parameterized by its `if` attribute—a two-argument object that returns either its first argument (for "true") or its second one (for "false"). The two objects `true` and `false` are simply two applications of `bool`, with opposite implementations of `if`.

Every boolean also has `not`, `and`, and `or` attributes.

## 5 Number

The object `number` is an abstraction of a floating point number of eight bytes according to 754 [2]. The bytes are encapsulated as the `as-bytes` attribute.

The `eq` attribute is `true` if the `as-bytes` is equal to another object, through the use of its `eq`.

The `⋆plus` attribute is a sum of this number and another `number`. The `minus` attribute is the subtraction of another `number` from this number. The `⋆times` attribute and `⋆div` are multiplication and division of this number and another `number`. The `neg` attribute is the same number with a different sign. The `floor` attribute is a rounded down `number`.

The `⋆gt` attribute is `true` if it is "greater than" another `number` and `false` otherwise. Attributes `lt`, `lte`, and `gte` are "less than," "less than or equal," and "greater than or equal" comparisons respectively.

The `is-nan`, `is-finite`, and `is-integer` attributes are predicates about the number, while `as-i64`, `as-i32`, and `as-i16` convert it to fixed-width integers.

Special values are provided as separate objects: `nan` for "not a number," together with `positive-infinity` and `negative-infinity`, also available as the short constants `pinf` and `ninf`.

## 6 String

The `string` object is an abstraction of a piece of Unicode text in UTF-8 encoding, according to 10646 [1]. UTF-8 uses one byte for the first 128 code points, and up to 4 bytes for other characters. The text is encapsulated as the `as-bytes` attribute.

The `⋆slice` attribute takes a piece of a string as another `string`. The `length` attribute is a total count of characters in the string.

Strings in other encodings, such as UTF-16 or CP1251, may be implemented through direct manipulations with `bytes`.

## 7 Tuple

The `tuple` object is an abstraction of an immutable sequence of objects. For example, this code represents a tuple `x` of three numbers:

```
* 65535 0xF21 3.14 > x
```

The `with` attribute is a new `tuple` with all elements from the current tuple and a new element added to the end of it. The `at` attribute is the element of the tuple at the specified index—positive indices are zero-based starting from the left, and negative indices start at -1 from the right. The `length` attribute is the total number of elements in the tuple.

## 8 Error Handling

The `⋆error` object terminates the program with an error. A copy is made with an encapsulated object as the error message; the first attempt to dataize it leads to a runtime error and program termination.

The `⋆try` object is the only mechanism able to catch such an error. It expects three arguments: `main`, `catch`, and `finally`. When dataized, it first attempts to dataize `main`; if an error occurs, the error is caught and `catch` is dataized instead, receiving the error as its only argument; the `finally` object is dataized afterwards in either case. This code prints a message instead of crashing on the division by zero:

```
try
  10.div 0
  io.stdout "cannot divide" > [e]
  true
```

## 9 Flow Control

The `seq` object is an abstraction of sequence of objects to be dataized sequentially. For example, this code prints one message to the console and then terminates the program due to the inability to dataize the division by zero in the middle:

```
seq *
```

```
  io.stdout "Hello, world!"
  42.div 0
  io.stdout "Bye!"
```

The objects `true` and `false` both have an attribute `if` which is a branching mechanism, expecting two arguments: a "positive" object, and a "negative" object. When being dataized, the object `true.if` always returns the "positive" object, the object `false.if` always returns the "negative" object. This code gives a title to a random number:

```
ms.random.pseudo > r
io.stdout
  tt.sprintf *1
    "Coin toss: %s"
    if.
      r.gte 0.5
      "heads"
      "tails"
```

The `while` object is an iterating mechanism, expecting two arguments. Both are abstract objects with one free attribute for current iteration index. The first argument is a condition, the second one is the body. When being dataized, the object `while` repeatedly dataizes the body while the `if` of the condition goes "positive." In the end, it returns the body or `false` if no iterations happened. This code makes a few attempts to find a random number that is smaller than 0.1:

```
ms.random.pseudo > r
while
  r.gte 0.1 > [i] >>
  io.stdout > [i]
    tt.sprintf *1
      "Attempt #%d"
      i
```

The `go` object enables forward and backward "jumps" either immediately finishing dataization or restarting it. For example, this code won't print a message to the console if `x` is equal to zero:

```
go.to
  seq * > [g]
    if > y
      x.eq 0
      g.forward 0
      42.div x
    io.stdout
      tt.sprintf *1
        "42/x = %d"
```

```
      y
```

In this example, the number is read from the console again, if it is equal to zero:

```
go.to
  seq * > [g]
    at. > x
      tt.sscanf
        "%d"
        io.stdin.next-line
      0
    if
      x.eq 0
      g.backward
      io.stdout
        tt.sprintf
          "Number %d is OK!"
          * x
```

The `switch` object is an abstraction of multi-case branching that encapsulates a `tuple` of $(k, v)$ pairs; the dataization result is equal to $v$ of the first pair where $k$ is `true`, or `true` if no pair matches:

```
io.stdout
  switch *
    *
      password.eq "swordfish"
      "password is correct!"
    *
      password.eq ""
      "empty password is not allowed"
    *
      false
      "password is wrong"
```

## 10 Digits

The `i16`, `i32`, and `i64` objects are decorators of `bytes` with a predefined size. They implement the same numeric operations as `number`, and can be converted to `number` through the ⋆`as-number` attribute. For example, the number 42 as a 64-bit integer always occupies exactly eight bytes, regardless of its value:

```
42.as-i64.as-bytes
```

The result is `00-00-00-00-00-00-00-2A`.

## 11 Memory

The `malloc` object is an abstraction of a block of random-access memory. A copy is created through one of its attributes: `of` allocates a block of a given size, `empty` allocates an empty block, and `for` allocates a block sized to hold a given object. The allocated block behaves like `bytes`, but additionally has `⋆write`, `⋆read`, `copy`, and `put` attributes. In this example, one kilobyte of memory with random data is allocated when a copy of `malloc` is created, six bytes are written starting at the 200th byte, and then a string is read back and `"Hello!"` is printed:

```
malloc.of
  1024
  seq * > [m]
    m.write 200 "Hello!"
    io.stdout
      string
        m.read 200 6
```

The `malloc` object also supports resizing via the `⋆resized` attribute and copying the data within the allocated memory block via the `copy` attribute. In the next example, `malloc` is allocated and filled with 5 bytes, then resized to 10 bytes and its content is duplicated. The result block contains `"hellohello"`.

```
malloc.of
  5
  seq * > [m]
    m.write 0 "hello"
    m.resized 10
    m.copy 0 5 5
```

The result of dataization of `malloc` is the result of dataization of its second argument, which is the scope where the memory block is allocated and cleared automatically when dataization has happened.

## 12 Math

All objects in this Section belong to the `ms` package. The `random` object is a pseudo-random number generator parameterized by a `seed`; when dataized it behaves as a `number` between zero and one. The `next` attribute is the next generator in the sequence, the `clocked` attribute seeds the generator from a given clock, and the `pseudo` attribute seeds it from the system clock.

The `angle` object is a decorator of `number`. It assumes that the angle is in radians and has the following attributes for trigonometric functions: `⋆sin`, `⋆cos`, `tan`, and `ctan`. The `degrees` attribute converts the angle from radians to degrees, and the `radians` attribute converts it back.

The constants `pi` and `e` approximate $\pi$ and Euler's number. The `real` object is a decorator of `number`. The `numbers` object decorates a `tuple` of numbers and provides `max` and `min` attributes. The `integral` object calculates the definite integral of a

function between two limits. For example, this code approximates the integral of $f(x) = x$ from one to ten with fifteen partitions, yielding approximately 49.5:

```
ms.integral
  x > [x]
  1
  10
  15
```

A few more decorators of `number`:

- `abs`: absolute value of the number
- `exp`: Euler's number raised to the given power
- `mod`: remainder of the division by another number
- ⋆`sqrt`: square root ($\sqrt{\rho}$)
- ⋆`pow`: the number raised to a given power ($\rho^x$)
- ⋆`ln`: natural logarithm ($\ln \rho$)
- ⋆`acos`: arccosine
- ⋆`asin`: arcsine

## 13 Texts

All objects presented in this Section belong to the `tt` package.

The ⋆`sscanf` object encapsulates two `string` objects: a format and a content. It behaves like a tuple of scanned data objects. For example, this code parses a hexadecimal number from the console:

```
at.
  tt.sscanf
    "%x"
    io.stdin
  0
```

The ⋆`sprintf` object builds a string according to the format provided and compliant with IEEE 1003.1 [13, Chapter 5], for example:

```
tt.sprintf *1
  "Hi, %s! The weight is %0.2f."
  "Jeff"
  3.14
```

A few more decorators of `string`:

- `trimmed`: removes both leading and trailing spaces
- `trimmed-left`: removes leading spaces
- `trimmed-right`: removes trailing spaces
- `joined`: joins a tuple of strings with this one as a glue
- `chained`: concatenates itself with other strings
- `repeated`: repeats itself a given number of times
- `slice`: takes a piece of itself as another string

- `at`: retrieves the symbol at a given index
- `contains`: checks whether it contains a substring
- `contains-all`: checks whether it contains all given substrings
- `contains-any`: checks whether it contains any given substring
- `starts-with`: checks whether it starts with a substring
- `ends-with`: checks whether it ends with a substring
- `index-of`: finds the first occurrence of a substring
- `last-index-of`: finds the last occurrence of a substring
- `up-cased`: makes it upper case
- `low-cased`: makes it lower case
- `replaced`: finds and replaces a substring
- `split`: breaks it into a tuple of strings
- `nsplit`: breaks it into at most a given number of strings
- `as-number`: converts itself to `number`
- `as-ascii`: reads a single character as its ASCII code
- `is-alpha`: checks whether all characters are letters
- `is-digit`: checks whether all characters are digits
- `is-alphanumeric`: checks whether all characters are letters or digits
- `is-ascii`: checks whether all characters are ASCII

The `string-buffer` object allows building a string incrementally, appending pieces through its `with` attribute. For example, this code builds the string `"Hello, Jeff"` and afterwards behaves as a plain `string`:

```
tt.string-buffer "Hello"
  .with ", "
  .with "Jeff"
```

The `regex` object is an abstraction of a regular expression in Perl-compatible format, with full support of Unicode. The `matches` attribute is `true` if the given text matches the pattern. The `match` attribute is a matcher over the given text: its `next` attribute is the first matched block, each block exposes `text`, `start`, and `group`, and its own `next` attribute is the following block.

```
tt.regex "/[a-z]+/" > r
r.match "!hello!world!" > m
eq.
  m.next.text
  "hello"
```

## 14 Structs

All objects presented in this Section belong to the `ss` package.

The object `list` is a decorator of `tuple` that adds a collection API. The attribute `is-empty` is `true` if the length of the tuple is zero. The attribute `eq` is `true` if both lists have the same length and each element is equal to the corresponding element of another list. The attribute `with` is a new list with a new element appended to the end

of it, while `withi` inserts a new element at the $i$-th position. The attribute `without` is a new list with all elements equal to the given one removed. The attribute `each` dataizes a given object for every element. The `front` and `back` attributes are the first and the last $N$ elements, `slice` is a sub-list between two indices, and `shifted` is the list without its first $N$ elements. The `sorted` attribute is a new list with elements sorted using the `lt` attribute of the elements. The `concat` attribute is a new list that concatenates the current list with the one provided as an argument.

The functional operations over a list are standalone objects of the `ss` package, each taking the list as its first argument. The objects `reduced`, `mapped`, `filtered`, and `eachi` are respectively similar to `reduce()`, `map()`, `filter()`, and `forEach()` methods of the `Array` object in JavaScript [11]. The "twin" objects `reducedi`, `mappedi`, and `filteredi` are semantically the same, but pass an extra `number` index to their function. The `contains`, `index-of`, and `last-index-of` objects search a list for an element, while `withouti` is a new list with the $i$-th element removed. The `bytes-as-array` object represents a sequence of `bytes` as a `tuple` of one-byte sequences. For example, this code doubles every element of a list, keeps the ones greater than four, and sums the survivors, resulting in 14:

```
ss.reduced
  ss.filtered
    ss.mapped
      ss.list (* 1 2 3 4)
      x.times 2 > [x]
    x.gt 4 > [x]
  0
  a.plus x > [a x]
```

The object `map` is a hash map built from a `tuple` of `map.entry` objects which are pairs $(k, v)$. The `map` ensures that all $k$ are always unique. It's expected that each $k$ is dataizable so it's possible to calculate `hash-code-of` it. The `with` attribute is a new map with a pair added or replaced, and `without` is a new map with a key removed. The `found` attribute tries to find an object in the map by the given key. The returned object has two attributes: `exists` which is `true` if the object was found; and `get` which is the found object or `⋆error`. The `keys` attribute is the `list` of map keys. The `values` attribute is the `list` of map values. The `size` attribute is the `size` of the map. The `has` attribute is `true` if the map contains the given *key*. For example, this code builds a map of two entries and reads the value 2 back through its key:

```
ss.map > m
  *
    ss.map.entry "one" 1
    ss.map.entry "two" 2
(m.found "two").get
```

The object `set` is an unordered collection of unique objects, built on top of `map`. It's expected that each element is dataizable so it's possible to calculate `hash-code-of` it.

Its `with` and `without` attributes add and remove an element, `has` checks membership, and `size` is the number of elements.

The object `hash-code-of` is the pseudo-unique floating-point numeric representation of an object. It's expected that the given object is dataizable.

The object `range` is a `list` that contains a range of elements. It accepts two attributes: `start` and `end`. The attribute `start` must be an abstract object that must have an attribute `next` to get the next element and an attribute `lt` to compare with the `end` object. Every next element also must have attributes `next` and `lt`. The first object in the chain must have a default value.

The object `range-of-numbers` is a `range` of integer numbers from `start` to `end` (soft border) with step = 1. It accepts two void attributes `start` and `end` which must be integer `number`s. If they're not, an `⋆error` is returned. For example, `ss.range-of-numbers 1 10` is the list `* 1 2 3 4 5 6 7 8 9`, since the upper bound is excluded.

## 15 I/O Streams

All objects presented in this Section belong to the `io` package.

It is expected that an input stream has the following interface:

```
[] > input
  [size] > read /bytes
```

It is expected that an output stream has the following interface:

```
[] > output
  [bytes] > write /true
```

The `console` object is a basic I/O object that allows interaction with the operating system console. It behaves as "input" and "output." The `stdout` object is an "output" that prints a `string` to the standard output stream. The `stdin` object is an "input" and an abstraction of a `string` currently available in the standard input stream. The attribute `next-line` reads a line until the EOL character (`\n`). The attribute `all-lines` reads all lines as long as possible. If there is no string in the stream, the object blocks dataization and waits. This code endlessly reads strings from the console and immediately prints them back:

```
while
  true > [i]
  io.stdout > [i]
    io.stdin.next-line
```

The object `bytes-as-input` is an "input" created from `bytes`. The object `malloc-as-output` is an "output" directed towards a copy of `malloc`. The object `tee-input` is an "input" and a channel between an "input" and an "output," which moves all available bytes from the former to the latter when being dataized. For example, this code writes text to a temporary file:

```
(fs.file "/tmp/foo.txt").open
```

```
[f]
  io.tee-input
    io.bytes-as-input
      "Hello, world!".as-bytes
    f
```

The object `input-length` is an object which reads all the bytes from the provided "input" and returns its length. The object `input-as-bytes` reads all the bytes from the provided "input" and returns them as `bytes`. The object `copied` copies all the bytes from an "input" to an "output" and returns the number of bytes copied. The objects `dead-input` and `dead-output` are stubs which read from and write to nothing.

## 16 File System

Here we define manipulations with files and directories. All objects presented in this Section belong to `fs` package.

The object `path` is an abstraction of file path and is a decorator of `string`. The attribute `separator` is a system-dependent default name-separator character. The attribute `joined` is a utility object that joins the given `tuple` of paths with the current OS separator and normalizes the result path. The attribute `as-file` is a `file` created from the path itself. The attribute `as-dir` is a `dir` created from the path itself. The attribute `is-absolute` is `true` if the path is absolute and `false` otherwise. The attribute `normalized` is a new `path` with normalized *uri*. Normalization includes converting multiple slashes into a single slash and resolving "." (current directory) and ".." (parent directory) segments. The attribute `resolved` is a new `path` with a suffix appended to the current one. The attribute `basename` is the name of the file in the path. The attribute `extname` is the extension in the path. The attribute `dirname` is the name of the directory in the path.

The object `file` is an abstraction of a file and is a decorator of `path`, which is the path of the file. The attribute `as-path` is a `path` of the file. The attribute ⋆`is-directory` is `true` if the file is a directory. The attribute ⋆`exists` is `true` if the file exists. The attribute `touched` makes sure the file exists. The attribute `deleted` removes the file. The attribute ⋆`size` is the size of the file in bytes. The attribute `moved` renames or moves the file to a new place. The attribute `open` opens the file. The `open` object accepts two attributes: file open mode which is a `string` and file scope. The file scope attribute must be an abstract object with one void attribute which is `file-stream`. It has the attributes ⋆`read` and ⋆`write` which allow working with the file as "input" and "output". The result of dataization of `file.open` is the result of dataization of its second argument "scope". The file descriptor is closed automatically when the scope is dataized.

The object `dir` is an abstraction of a directory. The attribute `made` creates the directory with all its parent directories. The attribute `deleted` deletes the directory recursively with all its subdirectories. The attribute ⋆`walk` finds files in the directory

using glob pattern matching. The attribute `tmpfile` is a temporary file in the directory.

The object `tmpdir` is a decorator of `dir` that abstracts the system directory for temporary files.

In the next example, a temporary file is created and filled up with `"Hello, world"`. Then the data is read from the file and written to `malloc`.

```
malloc.of
  12
  seq > [m]
    *
      tmpdir
      .tmpfile
      .open
        "w"
        f.write "Hello, world" > [f]
      .open
        "r"
        m.put > [f] >>
          f.read 12
      m
```

## 17 Net

Objects used for both server-side and client-side TCP sockets work in accordance with IEEE 1003.1 [13]. All objects presented in this Section belong to the `nk` package.

The `socket` object is an abstraction of a TCP socket. The `connect` attribute establishes a connection. It accepts an abstract object with one void attribute which is the socket descriptor. The socket descriptor has the following attributes:

- `send`: send a message to the socket
- `recv`: receive $x$ amount of bytes from the socket
- `as-input`: make "input" from socket
- `as-output`: make "output" from socket

When you dataize `socket.connect`, it creates a new socket, connects to the given IP on the given port, dataizes the provided scope, closes the socket and returns bytes retrieved from scope dataization. An `⋆error` is returned if any of the described steps failed.

This code opens a connection to TCP port 80 of `localhost` and then reads the entire stream as a Unicode string:

```
(nk.socket "127.0.0.1" 80).connect
  io.tee-input > [s]
    s.as-input
    io.console
```

The `listen` attribute allows you to listen to incoming connections as a server. It accepts an abstract object with one void attribute which is the socket descriptor. The current socket descriptor has the same attributes as the descriptor from `socket.connect` and also one extra attribute `accept`. This attribute is used to wait for an incoming connection. It also accepts an abstract object with one void attribute which is the client socket descriptor, which was connected to your server.

This code emulates a server on TCP port 8080 of `localhost`, then it accepts an incoming connection and sends `"Hello, world"` to it:

```
(nk.socket "127.0.0.1" 8080).listen
  s.accept > [s]
    client.send "Hello, world" > [client]
```

## 18 System

All objects presented in this Section belong to the `sm` package. They provide access to the underlying operating system.

The `os` object represents the current operating system. The `⋆name` attribute is its name, while `is-windows`, `is-linux`, and `is-macos` are predicates about its family.

The `getenv` object reads an environment variable as a `string`; an empty string means the variable does not exist.

The `eol` object, also available as `line-separator`, is the system-dependent line separator: `\n` on UNIX and `\r\n` on Windows.

The `system-clock` object is a wall-clock time source backed by the operating system; it decorates an `i64` holding the number of milliseconds since the Unix epoch.

The `posix` object makes a Unix system call by name through the POSIX interface, while the `win32` object makes a `kernel32.dll` function call by name; both accept a `tuple` of arguments.

## References


[1] ISO 10646. 2020. Information Technology — Universal Coded Character Set (UCS).

[2] IEEE 754. 2019. IEEE Standard for Floating-Point Arithmetic. doi:10.1109/IEEESTD.2019.8766229

[3] Brad Abrams and Tamara Abrams. 2005. *.NET Framework Standard Library Reference: Networking Library, Reflection Library, and XML Library.* Addison-Wesley Professional. doi:10.5555/1051166

[4] Apple. 2024. Swift Standard Library. https://jttu.net/swift2024. [Online; accessed 09-04-2024].

[5] Jim Blandy, Jason Orendorff, and Leonora Tindall. 2021. Programming Rust: Fast, Safe Systems Development.

[6] Grady Booch and Michael Vilot. 1990. The Design of the C++ Booch Components. In *Proceedings of the European Conference on Object-Oriented Programming Systems, Languages, and Applications*. 1–11. doi:10.1145/97945.97947

[7] Yegor Bugayenko and Maxim Trunnikov. 2021. $\varphi$-Calculus: Object-Oriented Formalism. arXiv:2111.13384 [cs.PL]

[8] C++. 2024. C++ Standard Library. https://en.cppreference.com/w/cpp/header. [Online; accessed 09-04-2024].

[9] Douglas Crockford. 2008. *JavaScript: The Good Parts*. Oreilly & Associates Inc. doi:10.5555/1386753
[10] Paul Deitel and Harvey Deitel. 2015. *Swift for Programmers*. Pearson PTR. doi:10.5555/2756809
[11] ECMA. 2011. Standard ECMA-262 — ECMAScript Language Specification.
[12] Doug Hellmann. 2017. *The Python 3 Standard Library by Example (Developer's Library)*. Addison-Wesley Professional. doi:10.5555/3161131
[13] IEEE 1003.1. 2018. IEEE Standard for Information Technology — Portable Operating System Interface (POSIX™) Base Specifications. doi:10.1109/IEEESTD.2018.8277153
[14] Nicolai Josuttis. 2012. *The C++ Standard Library: A Tutorial and Reference*. Addison-Wesley Professional. doi:10.5555/2544010
[15] Nikolai Kudasov and Violetta Sim. 2022. Formalizing $\varphi$-Calculus: A Purely Object-Oriented Calculus of Decorated Objects. In *Proceedings of the 24th International Workshop on Formal Techniques for Java-Like Programs*. 29–36. doi:10.1145/3611096.3611103
[16] Microsoft. 2023. .NET Standard. https://jttu.net/net2023. [Online; accessed 09-04-2024].
[17] Mozilla. 2024. Standard Built-in Objects. https://jttu.net/js2024. [Online; accessed 09-04-2024].
[18] Oracle. 2024. Java Development Kit, Version 22. https://openjdk.org/projects/jdk/22/. [Online; accessed 09-04-2024].
[19] Python. 2024. The Python Standard Library. https://docs.python.org/3/library/index.html. [Online; accessed 09-04-2024].
[20] Rust. 2024. The Rust Standard Library. https://doc.rust-lang.org/std/. [Online; accessed 09-04-2024].
[21] Herbert Schildt. 2018. *Java: The Complete Reference, Eleventh Edition*. McGraw-Hill Education.